

只用前向传播，就能微调大模型

普林斯顿提出 MeZO：在推理级显存下微调百亿参数模型（NeurIPS 2023）

微调（fine-tuning）是把一个通用预训练模型变得真正好用的关键一步，但它有一个越来越棘手的代价：显存。根源在于反向传播（backpropagation）。为了算出梯度，标准训练流程要在前向过程中缓存所有中间激活值，再在反向过程中存下一份完整梯度和一整套优化器状态。三者叠加，训练一步所需的显存最多可以达到同一模型做推理时的 12 $\times$ 。模型小的时候没人会在意，可一旦参数量涨到几百亿，这个倍数就成了你最先撞上的那堵墙。

换成硬件的语言会更直观。在一块 80GB 的 A100 上，你可以从容地对一个 30B 参数的模型做推理；但如果想用 Adam 在同一块卡上微调，能塞下的模型就只剩大约 2.7B，整整小了一个数量级。让人别扭的地方正在于此：硬件明明装得下、也跑得动这个大模型，只是负担不起基于梯度的训练额外压上来的那一大堆记账开销。

这篇来自普林斯顿大学（Princeton University）的论文追问的正是：这些记账开销真的不可省吗？它给出的答案是 MeZO，一种省显存的零阶优化器（zeroth-order optimizer），只用前向传播来微调语言模型，完全不做反向传播。核心结论是：这样训练所需的显存和推理几乎一样，却仍能在很多任务上追平基于梯度的微调。图 1 先把赌注摆清楚：在 OPT-13B 的 11 个任务上，MeZO 紧贴着完整的 Adam 微调，用的显存却只有它的 1/12，几乎回到了推理时的水平。

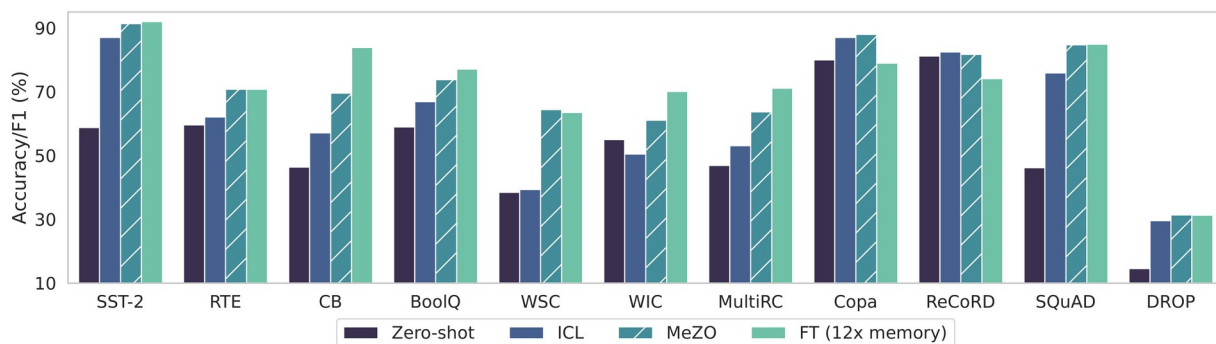


图 1. OPT-13B 上的 11 个任务：zero-shot、上下文学习（ICL）、MeZO，以及用 Adam 做的完整微调（FT）。MeZO 明显优于 zero-shot 和 ICL，并在 11 个任务里有 7 个与微调相差约 1% 以内，而显存只用了 1/12。

论文：<https://arxiv.org/abs/2305.17333> | 代码：<https://github.com/princeton-nlp/MeZO>

微调为什么这么吃显存

先把两种开销分开看。推理只需要模型权重，外加一小块给当前 token 用的激活缓冲。反向传播则在此之上多压了三样东西：整个前向过程缓存下来的激活值、一份和参数一样大的梯度，以及优化器状态——对 Adam 来说，那又是两份参数大小的滑动平均。加起来，一步训练的显存

占用就是一次前向传播的好几倍。像 LoRA、prefix tuning 这样的参数高效方法能压缩优化器和梯度那部分，却并没有去掉激活缓存，而且它们照样要做反向传播。

有一个绕开反向传播的老思路：零阶优化（zeroth-order optimization），只用函数值来估计梯度。你对参数做一点扰动，看损失如何变化，再从这两个数值里推断出一个下降方向。它一直被大模型冷落的原因，是一条颇为悲观的经典理论：零阶方法的收敛速度会随参数量成正比地变差。对一个动辄几十亿参数的模型来说，这听上去毫无希望。而本文最核心的经验结论恰恰是：当你是 在一个强预训练模型上、针对下游任务做微调时，这份悲观并不成立。

MeZO：塞进推理显存里的梯度估计

MeZO 建立在一个经典估计器 SPSA（同时扰动随机逼近，simultaneous perturbation stochastic approximation）之上。每一步采样一个与参数形状相同的随机方向 z ，服从标准高斯分布；给每个参数加上一小步 $\epsilon \cdot z$ 后记录损失，再减去这一步得到另一侧的损失。这两个损失之差除以 2ϵ ，就是一个标量——投影梯度；参数更新只需沿着 z 、按这个标量和学习率缩放后移动即可。

具体来说，SPSA 的梯度估计为 $\hat{\nabla}L(\theta) = [L(\theta + \epsilon z) - L(\theta - \epsilon z)] / (2\epsilon) \cdot z$ ，其中 $z \sim N(0, I)$ ；随后的参数更新就是普通的 SGD 一步 $\theta \leftarrow \theta - \eta \cdot \hat{\nabla}L(\theta)$ ，只沿着这一个随机采样出来的方向移动。

照字面实现，这仍会让显存翻倍，因为你得把完整的扰动向量 z 存下来，才能加上去、评估、再减回来。让 MeZO 真正可行的工程巧思，是干脆不存 z ：只保存那颗随机种子，在每次需要时按种子即时重新生成 z ——一次用于正向扰动，一次用于反向扰动，一次用于施加更新。因为同一颗种子会复现出同一个 z ，MeZO 得以就地对权重扰动再复位，于是整个「扰动—评估—更新」的循环都发生在一次普通前向传播的显存之内。每步只要两次前向传播，没有梯度，没有激活缓存，也没有优化器状态。

把省显存这件事测出来

图 3 把收益摆得很实在。它在 MultiRC 上、对从 1.3B 到 30B 的各档 OPT 模型测了 GPU 显存，比较 zero-shot 推理、上下文学习、完整微调、prefix-tuning 微调以及 MeZO。每一档规模上，MeZO 的柱子都紧贴在推理柱之上，而微调的柱子高高地压在上面。节省幅度随模型增大而增大，在 13B 上最高可达 12×；也正是它，让一块 80GB 的 A100 能微调 30B 的模型，而反向传播在同一块卡上只装得下 2.7B，差距悬殊得近乎荒诞。

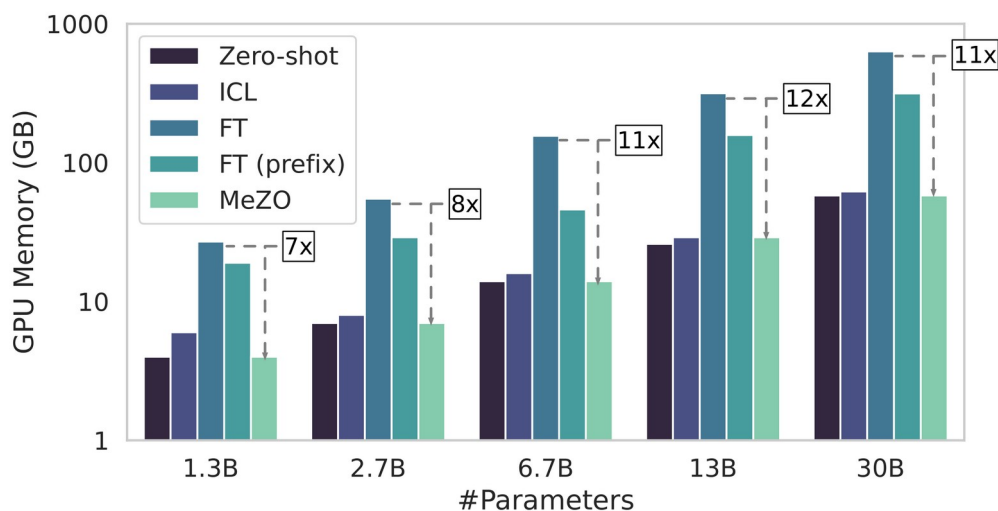


图 3. MultiRC（每条样本约 400 个 token）上不同 OPT 规模与调参方法的 GPU 显存。MeZO 用的显存和推理一样，相比标准微调最高可省 12×。

表 1. 按 OPT 模型规模看，MeZO 相比完整微调（Adam）的显存节省，数据读自图 3。MeZO 自身的显存在各档规模上都等同于推理。

OPT 模型	相比微调的节省
1.3B	7×
2.7B	8×
6.7B	11×
13B	12×
30B	11×

它真的能追平微调吗

省显存只有在质量站得住时才有意义，所以评测铺得很广。在自回归一侧，用了从 1.3B 一直到 66B 的 OPT 模型，覆盖 SuperGLUE 分类任务、COPA 与 ReCoRD 这类多选任务，以及 SQuAD、DROP 这类生成任务。在掩码语言模型一侧，用 RoBERTa-large（350M 参数）跑六个句子分类与推理任务，同时测少样本（ $k=16$ ）和多样本（ $k=512$ ）两种设置。对照组也都很诚实：包括 zero-shot、上下文学习、线性探测（LP），以及用 Adam 做的完整微调，几种常见做法都摆在一起比较。

图 1 里的 OPT-13B 结果是最凝练的一张总结。MeZO 明显胜过 zero-shot、上下文学习和线性探测，并在 11 个任务里有 7 个与完整微调相差约 1% 以内，而显存只有 1/12。RoBERTa-large 上，同样的规律在更小的规模上重演：MeZO 紧贴微调，在多样本 $k=512$ 设置下相差约 5%。而且这份省显存在实践中并不以时间为代价——在作者的实现里，微调一个 30B 模型时，MeZO 所需的 GPU 小时数比 Adam 微调还少约 2×。

它能和你已经在用的技巧叠加

MeZO 不是参数高效微调的替代品，而是叠在它之上。图 2 给出 RoBERTa-large 上把 MeZO 分别用于全参数、LoRA 和 prefix-tuning 的结果，并与各自的微调版本并排比较。几个 MeZO 变体彼此靠得很近，也都贴近微调——这意味着你可以把「只用前向传播」的训练和 LoRA 或 prefix-tuning 组合起来，同时保住精度。又因为 MeZO 全程只读取损失值、从不需要真正的梯度，它还能直接优化那些根本没有梯度的目标，比如直接优化准确率或 F1；而相较此前的零阶方法 BBTv2，它的准确率最高提升了 11%。

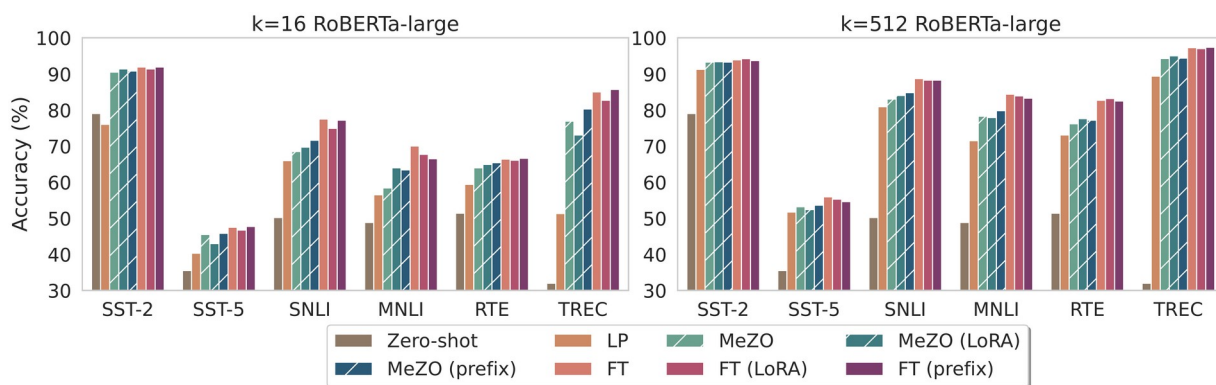


图 2. RoBERTa-large：zero-shot、线性探测（LP），以及全参数、LoRA、prefix-tuning 下的 MeZO 与微调（FT）。MeZO 胜过 zero-shot 和 LP，并以远更少的显存逼近微调（k=512 时相差约 5%）。

那份悲观错在哪里

论文还解释了：为什么一个被经典理论判定「应该慢如爬行」的方法，实际却能以可用的速度收敛。简单说，当你从一个训练得很好的模型出发时，当前权重附近的损失地形其实是近似低维的。MeZO 的收敛速度由这份局部有效结构——地形的局部秩——决定，而不取决于原始的参数量。这正是参数量悲观论不再咬人的原因：微调并不需要在完整的几十亿维空间里搜索，只需要在好解附近那一小块真正重要的子空间里搜索。

小结

最值得记住的一句话很简单：微调不再必须依赖反向传播。有了 MeZO，你可以只用前向传播来微调很大的语言模型，代价只是推理级的显存，却仍能在很多任务上追平基于梯度的质量；反过来，这也让你能在同样的硬件上训练大一个数量级的模型。边界也值得如实说清：MeZO 追平了微调的大部分差距，但并非全部；它可能需要更多步数才走到那里；而它之所以能快速收敛，前提是从一个足够强的预训练模型出发。在这些边界之内，它把推理的显存预算变成了训练预算，这是一笔实打实的好买卖。